

SecERJ: A Secure and Efficient Multi-party Protocol for Ripple Join

(Supplementary Material)

Chaobin Zheng

Beijing University of Posts and Telecommunications
Beijing, China
chaobinzheng@bupt.edu.cn

Zongda Han

Beijing University of Posts and Telecommunications
Beijing, China
jumdar@bupt.edu.cn

Xiang Cheng

Beijing University of Posts and Telecommunications
Beijing, China
chengxiang@bupt.edu.cn

Sen Su*

Beijing University of Posts and Telecommunications
Beijing, China
susen@bupt.edu.cn

A SECURITY PROOFS

In this appendix, we provide the formal ideal functionality and security proof for $\mathcal{P}_{\text{SecERJ}}$, followed by detailed security proofs for its three core sub-protocols: the Linear Batch Block Sampling protocol ($\mathcal{P}_{\text{LBS}}$), the Linear Secure Merge protocol ($\mathcal{P}_{\text{LME}}$), and the Linear Filter Sampling protocol ($\mathcal{P}_{\text{LFS}}$). All proofs follow the ideal/real paradigm under the semi-honest adversary model defined in Section 2.2. We assume that the underlying primitives: secret sharing, oblivious sort, PSI, permutation, prefix sum, segmented prefix sum, compaction, and expansion, are each UC-secure (universally composable secure) against semi-honest adversaries in the 3PC setting, as established by their respective references cited in Section 2.

Throughout this appendix, we write $\text{VIEW}_{P_i}^\Pi(\mathbf{x})$ to denote the view of party P_i during the execution of protocol Π on input $\mathbf{x}$, consisting of P_i 's input share, its random tape, and all messages it receives. We write $\equiv$ for computational indistinguishability.

A.1 Security of $\mathcal{P}_{\text{SecERJ}}$

Ideal Functionality $\mathcal{F}_{\text{SecERJ}}$.

Ideal functionality for Secure and Efficient Ripple Join

Inputs: $R = \{\llbracket R_1 \rrbracket, \dots, \llbracket R_K \rrbracket\}$; Sample size $s = \{s_1, \dots, s_K\}$;
Join aggregate query Q ; Confidence level α ; Expected error E
Outputs: Estimate $\llbracket T_n \rrbracket$, Bit vector $(0, 0, \dots, 0, 1)$

THEOREM A.1 (SEMI-HONEST SECURITY OF $\mathcal{P}_{\text{SecERJ}}$). *Under the assumptions of Theorems A.2–A.4 and the UC-security of $\mathcal{P}_{\text{Join}}$, $\mathcal{P}_{\text{SegSum}}$, $\mathcal{P}_{\text{Compt}}$, and the floor division protocol, the protocol $\mathcal{P}_{\text{SecERJ}}$ UC-securely realizes $\mathcal{F}_{\text{SecERJ}}$ against semi-honest adversaries corrupting at most one of the three servers.*

Proof (Sketch). We argue security by the UC composition theorem applied to the sequential execution of sub-protocols across all iterations. The sampling phase invokes either $\mathcal{P}_{\text{LBS}}$ or $\mathcal{P}_{\text{LFS}}$, both UC-secure by Theorems A.2 and A.4. In each iteration t , $\mathcal{P}_{\text{SecERJ}}$ sequentially invokes:

- (i) Selection of $\frac{s_i}{s}$ sample blocks from the pre-sampled batch (local array indexing on shares, no communication).

*Corresponding author.

- (ii) $\mathcal{P}_{\text{sort}}$ on the new blocks $\beta_{i,t}$; UC-secure by [?].
- (iii) $\mathcal{P}_{\text{LME}}$ to merge β into $\mathcal{B}$ and update merged positions; UC-secure by Theorem A.3.
- (iv) $\mathcal{P}_{\text{Join}}$ on the integrated sets $\mathcal{B}_{1,t}, \dots, \mathcal{B}_{K,t}$; UC-secure by [?].
- (v) Group-by aggregation via $\mathcal{P}_{\text{perm}}$, $\mathcal{P}_{\text{SegSum}}$, and $\mathcal{P}_{\text{Compt}}$, all of which are UC-secure.
- (vi) Computation of $\llbracket \mu \rrbracket$ and $\llbracket \epsilon \rrbracket$ via local arithmetic and the floor division protocol.
- (vii) Termination check: a secure comparison of $\llbracket \epsilon/\mu \rrbracket$ against the public threshold E , revealing only a single termination bit.

Each invocation is UC-secure. By the UC composition theorem, their sequential composition is also UC-secure. The only additional leakage is: (1) the termination bit vector $(0, \dots, 0, 1)$, which reveals the number of iterations required to meet the error bound; this is intentional and necessary, as the protocol cannot terminate without it; (2) the join result size $|U|$ at each iteration, which is a standard leakage in all known MPC join protocols [?]. This completes the proof sketch.

A.2 Security of $\mathcal{P}_{\text{LBS}}$

Ideal Functionality $\mathcal{F}_{\text{LBS}}$. The ideal functionality $\mathcal{F}_{\text{LBS}}$ takes as input a secret-shared relation $\llbracket R \rrbracket$ of size n and a public block size s , where n is a multiple of s (after padding). It proceeds as follows:

- (1) Reconstruct R internally.
- (2) Apply a uniformly random permutation $\sigma \leftarrow \mathfrak{S}_n$ to obtain $R' = \sigma(R)$.
- (3) Partition R' into $T = n/s$ consecutive blocks of size s .
- (4) Sample T block indices $I_1, \dots, I_T$ independently and uniformly at random from $\{0, \dots, T-1\}$ (sampling with replacement).
- (5) Output the secret sharing $\llbracket S \rrbracket$ of the concatenated selected blocks $R'[I_1] \parallel \dots \parallel R'[I_T]$, together with their sample identifiers $\llbracket \text{Sid} \rrbracket$.

The public output consists solely of the total output size $T \cdot s$, which equals the input size n and is non-sensitive.

THEOREM A.2 (SEMI-HONEST SECURITY OF $\mathcal{P}_{\text{LBS}}$). *Under the assumption that $\mathcal{P}_{\text{sort}}$, $\mathcal{P}_{\text{PSI}}$, $\mathcal{P}_{\text{SegSum}}$, and $\mathcal{P}_{\text{perm}}$ are each UC-secure*

against semi-honest adversaries corrupting at most one of the three servers, the protocol $\mathcal{P}_{LBS}$ UC-securely realizes $\mathcal{F}_{LBS}$ against semi-honest adversaries.

PROOF. We construct a simulator $\mathcal{S}_{LBS}$ for each possible corrupted party P_i ($i \in \{1, 2, 3\}$) and show that the simulated view is computationally indistinguishable from the real view.

Simulator $\mathcal{S}_{LBS}$. Given the corrupted party P_i 's input share $\llbracket R \rrbracket_{P_i}$ and the ideal output share $\llbracket S \rrbracket_{P_i}$, $\mathcal{S}_{LBS}$ proceeds as follows:

- Step 1.** *Simulating the shuffle (dummy padding and random permutation).* The shuffle at Line 1 of Protocol ?? is performed by the data owners in plaintext before secret-sharing, or equivalently realized by a random permutation applied locally. The dummy tuples added for padding are public constants. Thus P_i sees only its own share, which is uniformly random by the (3,2)-secret-sharing scheme. $\mathcal{S}_{LBS}$ generates a uniformly random share of the same length as the input and uses it as the simulated share for this step.
- Step 2.** *Simulating the index generation.* The block indices $\llbracket b_i(Bid) \rrbracket$ are generated by sampling a secret-shared random value and applying the secure modulo operation (Section 2.4). By the security of the modulo protocol (which reduces to secure multiplication and comparison, both UC-secure), the view of P_i during this step consists of uniformly random shares over $\mathbb{Z}_{2^k}$. $\mathcal{S}_{LBS}$ samples fresh uniform random shares of the same number and uses them as the simulated messages.
- Step 3.** *Simulating $\mathcal{P}_{sort}$ on b .* By the UC-security of $\mathcal{P}_{sort}$ [?], there exists a simulator $\mathcal{S}_{sort}$ that, given only the input and output share of P_i , produces a view computationally indistinguishable from the real view. $\mathcal{S}_{LBS}$ invokes $\mathcal{S}_{sort}$ internally.
- Step 4.** *Simulating the replication step.* The replication of b to form B (Lines 8–9 of Protocol ??) involves only local arithmetic on shares (scaling and addition of constants). The resulting messages at P_i are deterministic linear functions of its share of b . $\mathcal{S}_{LBS}$ replicates the simulated shares from Step 2 in the same manner.
- Step 5.** *Simulating $\mathcal{P}_{PSI}$.* By the UC-security of $\mathcal{P}_{PSI}$ [?], there exists a simulator $\mathcal{S}_{PSI}$ that produces an indistinguishable view given only the input and output shares of P_i . $\mathcal{S}_{LBS}$ invokes $\mathcal{S}_{PSI}$ with the simulated inputs and the corresponding share of the ideal output $\llbracket S \rrbracket$.
- Step 6.** *Simulating $\mathcal{P}_{SegSum}$ and $\mathcal{P}_{perm}$.* By UC-security of these primitives (referenced in Section 2.5), their simulators exist analogously. $\mathcal{S}_{LBS}$ invokes each internal simulator in sequence.

Indistinguishability argument. The composed simulator $\mathcal{S}_{LBS}$ constructs a view that is a sequential composition of the views produced by $\mathcal{S}_{sort}$, $\mathcal{S}_{PSI}$, $\mathcal{S}_{SegSum}$, and $\mathcal{S}_{perm}$, connected by local computations on shares. By the UC composition theorem [?], sequential composition of UC-secure protocols in a common reference model preserves UC-security. Therefore:

$$\mathcal{S}_{LBS}(P_i, \llbracket R \rrbracket_{P_i}, \llbracket S \rrbracket_{P_i}) \stackrel{c}{\equiv} \text{VIEW}_{P_i}^{\mathcal{P}_{LBS}}(R, s).$$

The only information that leaks structurally is the sizes n and $T \cdot s = n$, which are identical and public. Since the block indices, the shuffle permutation, and all intermediate values are fully secret-shared and never reconstructed, no single server gains any information about the data or the selected blocks beyond this public size. This completes the proof. $\square$

A.3 Security of $\mathcal{P}_{LME}$

Ideal Functionality $\mathcal{F}_{LME}$. The ideal functionality $\mathcal{F}_{LME}$ takes as input two secret-shared sorted tables $\llbracket R \rrbracket$ and $\llbracket S \rrbracket$ of sizes m and n respectively, joined on attribute B . It proceeds as follows:

- (1) Reconstruct R and S internally.
- (2) For each $r_i \in R$, let c_i be the number of elements in S with $s.B = r_i.B$. Assign the merged position $R_i(MB) = i + \sum_{k=1}^{i-1} c_k$.
- (3) For each $s_j \in S$: if $s_j.B$ matches some $r_i.B$, assign $S_j(MB) = R_i(MB) + \text{rank}(s_j)$, where $\text{rank}(s_j)$ is the rank of s_j among elements of S matching $r_i.B$; otherwise append s_j at position $|R| + |S| + 1 - \text{rank}'$, where rank' is the rank of s_j among unmatched elements.
- (4) Output $\llbracket R(MB) \rrbracket$ and $\llbracket S(MB) \rrbracket$.

The public output is the sizes m and n , which are non-sensitive (revealed by the join result size).

THEOREM A.3 (SEMI-HONEST SECURITY OF $\mathcal{P}_{LME}$). *Under the assumption that $\mathcal{P}_{SegSum}$, $\mathcal{P}_{PSI}$, $\mathcal{P}_{PrefixS}$, and $\mathcal{P}_{perm}$ are each UC-secure against semi-honest adversaries corrupting at most one server, the protocol $\mathcal{P}_{LME}$ UC-securely realizes $\mathcal{F}_{LME}$ against semi-honest adversaries.*

PROOF. We construct a simulator $\mathcal{S}_{LME}$ for the corrupted party P_i .

Simulator $\mathcal{S}_{LME}$. Given P_i 's input shares $\llbracket R \rrbracket_{P_i}$, $\llbracket S \rrbracket_{P_i}$ and the ideal output shares $\llbracket R(MB) \rrbracket_{P_i}$, $\llbracket S(MB) \rrbracket_{P_i}$, $\mathcal{S}_{LME}$ proceeds as follows:

- Step 1.** *Simulating $\mathcal{P}_{SegSum}(1_m, S(B), 0)$ to obtain $S(V)$.* This invokes the segmented prefix sum of an all-ones vector segmented by $S(B)$. Its purpose is to compute the within-group rank of each element of S . By UC-security of $\mathcal{P}_{SegSum}$ [?], the simulator $\mathcal{S}_{SegSum}$ produces an indistinguishable view. $\mathcal{S}_{LME}$ invokes $\mathcal{S}_{SegSum}$ with the input share of P_i and a freshly sampled uniform share as the simulated output share.
- Step 2.** *Simulating the local assignment of $R(O)$.* Each party assigns $\llbracket R_i(O) \rrbracket = \llbracket i \rrbracket$ using a local addition on a public counter and a zero sharing (via $\mathcal{P}_{zero}$). This involves no communication. $\mathcal{S}_{LME}$ replicates this step deterministically.
- Step 3.** *Simulating the first $\mathcal{P}_{PSI}(R, S, S(V))$ to get $R(V')$.* This PSI call queries, for each $r_i \in R$, the count $S(V)$ of the last matching element in S (or 0 if none). By UC-security of $\mathcal{P}_{PSI}$ [?], $\mathcal{S}_{LME}$ invokes $\mathcal{S}_{PSI}$ with the appropriate input and output shares.
- Step 4.** *Simulating the iterative computation of $R(MB)$ (Lines 5–6).* The loop computes a cumulative sum: $\llbracket C_i \rrbracket = \llbracket C_{i-1} + R_{i-1}(V') \rrbracket$ and $\llbracket R_i(MB) \rrbracket = \llbracket R_i(O) + C_i \rrbracket$. Both operations are local additions on shares (no communication).

Since the output share $\llbracket R(MB) \rrbracket_{P_i}$ is already provided by the ideal functionality, $\mathcal{S}_{LME}$ decomposes it into consistent simulated intermediate shares by sampling a uniform random $\llbracket C \rrbracket_{P_i}$ and setting the local addition accordingly.

Step 5. *Simulating the second $\mathcal{P}_{PSI}(S, R, R(MB))$ to get $S(g), S(f)$.* Here $R(MB)$ is used as the payload. By UC-security of $\mathcal{P}_{PSI}$, $\mathcal{S}_{LME}$ invokes $\mathcal{S}_{PSI}$ using the simulated $\llbracket R(MB) \rrbracket_{P_i}$ as the payload and fresh uniform shares as the simulated outputs.

Step 6. *Simulating $\mathcal{P}_{SegSum}(\{S(g), S(f)\}, S(B), 1)$ and $\mathcal{P}_{PrefixS}(1 - S(f), 0)$.* Both are UC-secure; $\mathcal{S}_{LME}$ invokes their respective simulators.

Step 7. *Simulating the computation of $S(MB)$ (Lines 10–12).* This involves only local multiplications and additions on shares. $\mathcal{S}_{LME}$ uses the ideal output share $\llbracket S(MB) \rrbracket_{P_i}$ directly, generating consistent intermediate values by backward substitution using random masks.

Indistinguishability argument. Each step above produces a simulated message distribution that is computationally indistinguishable from the real distribution, by the UC-security of the respective primitive or by the uniformity of (3,2)-secret sharing. The only structural leakage is the sizes $|R| = m$ and $|S| = n$, which are already known (they equal the sizes of the accumulated and new sample sets at each iteration, which are public by the join output size convention noted in Section 2.5). By the UC composition theorem:

$$\mathcal{S}_{LME}(P_i, \llbracket R \rrbracket_{P_i}, \llbracket S \rrbracket_{P_i}, \llbracket R(MB) \rrbracket_{P_i}, \llbracket S(MB) \rrbracket_{P_i}) \stackrel{c}{\equiv} \text{VIEW}_{P_i}^{\mathcal{P}_{LME}}(R, S).$$

We further note that the merged positions $R(MB)$ and $S(MB)$ are computed and retained entirely in secret-shared form throughout the iterative process of SecERJ. They are never reconstructed by any single server. The matching structure (i.e., which elements of S match which elements of R) remains hidden since PSI outputs only shares of the match results. The flag f indicating whether a match exists is also secret-shared and never revealed. This completes the proof. $\square$

A.4 Security of $\mathcal{P}_{LFS}$

Ideal Functionality $\mathcal{F}_{LFS}$. The ideal functionality $\mathcal{F}_{LFS}$ takes as input a secret-shared relation $\llbracket R \rrbracket$ of size n (padded to a multiple of s), a public block size s , and a selection predicate P whose attribute values are secret-shared. It proceeds as follows:

- (1) Reconstruct R internally. Evaluate the predicate P on each tuple to obtain validity bits $f \in \{0, 1\}^n$. Let $Tot = \sum_i f_i$.
- (2) Compact valid tuples to the front, and compute $H = \lceil Tot/s \rceil$ (the number of valid blocks), $M = H \cdot s - Tot$ (invalid tuples in the last valid block), and the redistribution counts $g_1, \dots, g_H$ that evenly spread the M invalid tuples across all valid blocks.
- (3) Apply the redistribution permutation G to obtain the filtered table.
- (4) Execute the LBS procedure (as in $\mathcal{F}_{LBS}$) with modulus H instead of T .
- (5) Output the secret sharing $\llbracket S \rrbracket$ and $\llbracket Sid \rrbracket$.

The public output is the total output size $T \cdot s = n$, which is non-sensitive.

THEOREM A.4 (SEMI-HONEST SECURITY OF $\mathcal{P}_{LFS}$). *Under the assumption that the comparison protocol, $\mathcal{P}_{Compt}$, $\mathcal{P}_{PrefixS}$, $\mathcal{P}_{perm}$, and $\mathcal{P}_{LBS}$ are each UC-secure against semi-honest adversaries corrupting at most one server, the protocol $\mathcal{P}_{LFS}$ UC-securely realizes $\mathcal{F}_{LFS}$ against semi-honest adversaries. In particular, neither the number of valid tuples Tot , the number of valid blocks H , nor the distribution of invalid tuples within valid blocks is revealed to any single server.*

PROOF. We construct a simulator $\mathcal{S}_{LFS}$ for the corrupted party P_i .

Simulator $\mathcal{S}_{LFS}$. Given P_i 's input share $\llbracket R \rrbracket_{P_i}$ and the ideal output share $\llbracket S \rrbracket_{P_i}$, $\mathcal{S}_{LFS}$ proceeds as follows:

- Step 1.** *Simulating predicate evaluation (Lines 2–5).* The validity bits $\llbracket R_i(f) \rrbracket$ are computed via a secure comparison protocol (Section 2.4), which is UC-secure. The accumulation $\llbracket Tot \rrbracket = \sum_i \llbracket R_i(f) \rrbracket$ uses only local additions. $\mathcal{S}_{LFS}$ invokes the simulator $\mathcal{S}_{cmp}$ for each comparison and produces a uniform random share as the simulated $\llbracket Tot \rrbracket_{P_i}$.
- Step 2.** *Simulating $\mathcal{P}_{Compt}(R, R(f))$ (Line 6).* By the UC-security of $\mathcal{P}_{Compt}$ (which is built from $\mathcal{P}_{permGen}$ and $\mathcal{P}_{perm}$), $\mathcal{S}_{LFS}$ invokes $\mathcal{S}_{Compt}$ with the input share and a fresh uniform output share.
- Step 3.** *Simulating the computation of H, M, M_1, M_2 (Line 7).* These are computed via the secure floor division and modulo protocols (Section 2.4). Each of these protocols is a composition of secure multiplication and comparison, both UC-secure. Since all computed values remain secret-shared, $\mathcal{S}_{LFS}$ simulates each sub-protocol call using its respective UC simulator, producing uniformly distributed shares.
- Step 4.** *Simulating the per-block allocation computation g_i (Lines 8–10).* The loop body involves two secure comparison calls ($\llbracket i \leq H \rrbracket$ and $\llbracket i \leq M_2 \rrbracket$) and local multiplications/additions. The comparisons are simulated via $\mathcal{S}_{cmp}$. The local operations produce messages that are deterministic functions of the share values, which the simulator computes consistently.
Critical observation: The index i in the loop ranges over public values $1, \dots, N_b$. The comparison $\llbracket i \leq H \rrbracket$ compares a public value against a secret-shared one, returning a secret-shared bit. No individual server learns which blocks are valid (i.e., $i \leq H$) or the exact value of H . The UC-security of the comparison protocol ensures that the simulated bit shares are indistinguishable from real ones.
- Step 5.** *Simulating $\mathcal{P}_{PrefixS}(g)$ (Line 11).* By UC-security, the simulator $\mathcal{S}_{PrefixS}$ produces an indistinguishable view.
- Step 6.** *Simulating the insertion position computation and $\mathcal{P}_{Compt}(R(e), 1 - R(fd))$ (Lines 12–17).* The nested loop computes insertion flags $\llbracket R_{i \cdot s + j}(fd) \rrbracket$ and positions $\llbracket R_{i \cdot s + j}(e) \rrbracket$ via secure comparisons and local arithmetic. The compaction call at Line 17 moves the e -values downward. All are UC-secure sub-protocols; $\mathcal{S}_{LFS}$ invokes their respective simulators.
- Step 7.** *Simulating the final permutation and $\mathcal{P}_{LBS}$ (Lines 18–20).* The permutation $\mathcal{P}_{perm}(R, R(G))$ is UC-secure by assumption. The subsequent $\mathcal{P}_{LBS}$ call is UC-secure by Theorem A.2. $\mathcal{S}_{LFS}$ invokes $\mathcal{S}_{perm}$ followed by $\mathcal{S}_{LBS}$ using the ideal output share $\llbracket S \rrbracket_{P_i}$.

Indistinguishability argument. By the UC composition theorem, the sequential composition of the above simulation steps is computationally indistinguishable from the real protocol execution:

$$\mathcal{S}_{LFS}(P_i, \llbracket R \rrbracket_{P_i}, \llbracket S \rrbracket_{P_i}) \stackrel{c}{\equiv} \text{VIEW}_{P_i}^{\mathcal{P}_{LFS}}(R, s, P).$$

We specifically address the three sensitive values identified in the theorem statement:

- *Number of valid tuples Tot.* Tot is computed as a sum of secret-shared bits and remains secret-shared throughout. It is never reconstructed.
- *Number of valid blocks H.* $H = \lceil \text{Tot}/s \rceil$ is derived from $\llbracket \text{Tot} \rrbracket$ via a secure floor division and remains secret-shared as $\llbracket H \rrbracket$. The random block indices in the subsequent LBS call use $\llbracket H \rrbracket$

as the secret-shared modulus. By security of the modulo protocol, the resulting indices $\llbracket \text{RAND} \% H \rrbracket$ are computationally indistinguishable from uniform shares.

- *Distribution of invalid tuples within valid blocks.* The per-block allocation g_i depends on $\llbracket H \rrbracket$, $\llbracket M_1 \rrbracket$, and $\llbracket M_2 \rrbracket$, all of which are secret-shared. The compaction and permutation operations are oblivious (data-independent in their communication patterns). Therefore, no server learns the arrangement of valid versus invalid tuples within any block.

The only structural information leaked is the total padded table size n and the output size $T \cdot s = n$, both of which are public. This completes the proof. $\square$